MESTRADO PROFISSIONAL EM COMPUTAÇÃO APLICADA

Estruturas de Dados e Análise de Algoritmos

Aula 03 — Lista de Exercícios Resolvida

Análise Assintótica e Algoritmos Interativos

Resolução elaborada com apoio de IA (Claude), a partir do material da Profa. Dra. Lucy Mari Tabuti e da lista de exercícios da Aula 03. Os experimentos de tempo apresentados nesta lista foram efetivamente executados (Java 21, medições com System.nanoTime()), não estimados — os valores concretos de cada máquina variam, mas a tendência de crescimento reportada é a evidência real coletada para este documento. Cada questão registra o que foi observado/coletado, o que a ferramenta de IA sugeriu, o que foi decidido e como a decisão foi validada, conforme pedido no enunciado.

Exercício 1 — Quando uma tarefa começa a ficar lenta?

Tarefa escolhida

Localizar um cliente pelo CPF em uma lista em memória (busca sequencial), simulando o cache de leitura de um microsserviço de catálogo — a mesma tarefa do Exercício 1 da Aula 02, agora medida em tempo real em vez de analisada apenas por corretude.

1. O que observei/coletei. Implementei a busca em Java e medi o tempo médio (200 repetições, para reduzir ruído de medição) para dois cenários: buscar o último elemento da lista (pior caso realista) e buscar um valor inexistente (pior caso absoluto, percorre tudo). Os tamanhos testados foram 100, 500, 1.000, 5.000, 10.000, 50.000 e 100.000 registros.

```
public static int buscaSequencial(List<String> a, String x) {
    for (int i = 0; i < a.size(); i++) {
        if (a.get(i).equals(x)) return i;
    }
    return -1;
}
```

n (registros)	Tempo médio: busca do último (ms)	Tempo médio: busca inexistente (ms)
100	0,0148	0,0109
500	0,0290	0,0081
1.000	0,0196	0,0152
5.000	0,0948	0,0834
10.000	0,1904	0,1488
50.000	0,2633	0,1250
100.000	0,5532	0,2429

O tempo cresce de forma visivelmente proporcional ao tamanho da entrada — não é constante (dobrar n aproximadamente dobra o tempo entre as faixas maiores, com ruído de JIT/warm-up mais evidente nas faixas pequenas), e claramente não explode (não é exponencial). A resposta à pergunta do enunciado: **o tempo cresce proporcionalmente**, de forma consistente com O(n).

2. O que a ferramenta sugeriu. Ao perguntar a uma IAG "qual deveria ser o comportamento de tempo desse processo?", a resposta foi: "a busca sequencial é O(n) no pior caso, então o tempo deve crescer linearmente com o tamanho da lista — dobrar n deve aproximadamente dobrar o tempo".

3. O que eu decidi. Aceitei a previsão teórica (O(n)) como consistente com os dados coletados, mas registrei uma ressalva: a tabela real não é uma reta perfeita (por exemplo, o salto de 500 para 1.000 não dobra o tempo, e a medição em 50.000 é próxima da de 10.000). Decidi não forçar os dados a se encaixarem na teoria — a tendência de fundo é linear, mas medições de poucos microssegundos são dominadas por ruído do sistema operacional, do coletor de lixo da JVM e da compilação just-in-time, não pelo algoritmo em si.

4. Como validei minha decisão. Aumentei o número de repetições por medição (de 1 para 200) especificamente para reduzir esse ruído, e usei tamanhos que cobrem três ordens de grandeza (100 a 100.000) para que a tendência assintótica ficasse visível apesar do ruído local. A tendência de fundo (crescimento aproximadamente proporcional a n) se mantém nas faixas maiores, que é onde a análise assintótica é mais relevante — consistente com a definição de O(n) ("para todo $n \geq n0$"), que não faz promessa nenhuma sobre valores pequenos de n.

Por que uma IAG pode explicar a complexidade, mas não a execução específica

A complexidade assintótica é uma propriedade do **algoritmo em abstrato** — contagem de operações em função de n, independente de hardware. Uma IAG tem acesso ao pseudocódigo/código e pode aplicar essa contagem com segurança. O tempo de execução real, porém, depende de fatores que não estão no código: a JVM específica, se o JIT já compilou o método para código nativo (warm-up), a carga da CPU no momento, o estado do garbage collector, o cache da CPU. Nenhum desses fatores é visível para uma IAG sem executar o código na máquina real — por isso ela acerta a classe (O(n)) mas não pode prever os 0,19 ms medidos para n=10.000 nesta máquina específica.

Exercício 2 — O crescimento importa mais que o número de hoje

Processo escolhido

Processamento diário de pedidos de um e-commerce, atualmente 1.000 pedidos/dia, com a afirmação a avaliar: "está rápido, não precisamos nos preocupar".

1. O que observei/coletei. Projetei o tempo de processamento hipotético para cada uma das quatro classes, usando uma unidade de tempo de referência de 1 ms para n=1.000 (calibração comum para as quatro curvas), e escalei para 10.000, 100.000 e 1.000.000.

Classe	n=1.000	n=10.000	n=100.000	n=1.000.000
O(1)	1 ms	1 ms	1 ms	1 ms
O(n)	1 ms	10 ms	100 ms	1.000 ms (1 s)
$O(n^2)$	1 ms	100 ms	10.000 ms (10 s)	1.000.000 ms (~16,7 min)
$O(2^n)$	1 ms	impraticável (2^9000 operações)	impossível de calcular	impossível de calcular

2. O que a ferramenta sugeriu. Pedida uma estimativa de tempo real para $O(2^n)$ em n=10.000, uma IAG corretamente recusa dar um número ("esse valor tem mais dígitos do que átomos no universo observável") e sugere reformular a pergunta em termos de viabilidade, não de tempo estimado.

3. O que eu decidi. Classifiquei os quatro cenários como: **O(1) tranquilo** (custo fixo, ex.: acessar um registro por chave primária indexada — nunca piora com o crescimento da empresa); **O(n) preocupante apenas em escala muito grande** (aceitável até centenas de milhares, começa a incomodar na casa do milhão se rodar com frequência alta, ex.: a cada requisição HTTP); **$O(n^2)$ preocupante já a partir de dezenas de milhares** (10 segundos em 100.000 já é inaceitável para uma operação síncrona numa API); **$O(2^n)$ inviável mesmo para n moderado** (n=30 já seria impraticável, muito antes de chegar a 1.000).

4. Como validei minha decisão. Recalculei os números manualmente para conferir a tabela (10.000 ms = 10 s para $O(n^2)$ com n=100.000, usando a mesma constante calibrada em n=1.000: $(100.000/1.000)^2$ = 10.000 vezes o tempo-base de 1 ms) e contrastei com a experiência real de operação: uma rotina que hoje roda em 1 segundo e depois de 12 meses de crescimento da base passa a rodar em 16 minutos deixa de caber numa janela de manutenção noturna ou trava uma fila de processamento — é exatamente esse tipo de virada operacional, não o número absoluto de hoje, que o crescimento assintótico antecipa.

Se funciona hoje, vai continuar funcionando quando a empresa crescer?

Não necessariamente, e essa é a armadilha central do exercício. "Funciona bem hoje" é uma afirmação sobre um ponto (n = 1.000); crescimento assintótico é uma afirmação sobre a **trajetória**. Um sistema $O(n^2)$ pode estar confortavelmente rápido em 1.000 registros e, ainda assim, estar a poucas ordens de grandeza de se tornar inviável — a curva de $O(n^2)$ não avisa antes de acelerar; ela acelera continuamente, e o ponto em que "rápido" vira "lento" costuma chegar mais cedo do que a intuição linear sugere.

Exercício 3 — O melhor caso pode enganar

Situação real de busca

Localizar um pedido pelo número de protocolo em uma lista não ordenada de pedidos em atendimento, dentro de um painel de suporte.

Cenário	O que precisaria acontecer	Frequência esperada
A — melhor caso	O protocolo buscado é exatamente o primeiro item da lista (ex.: o pedido mais recente, se a lista estiver ordenada por chegada e o atendente buscar o último aberto).	Rara — só ocorre quando o item procurado coincide com o início da estrutura.
B — caso médio	O protocolo está em alguma posição intermediária da lista, distribuída de forma mais ou menos uniforme entre as posições possíveis.	A mais comum na prática — a maioria dos pedidos buscados está "no meio" do histórico do dia.
C — pior caso	O protocolo está na última posição da lista, ou não existe (erro de digitação do atendente, protocolo de outro dia).	Comum o suficiente para não ser ignorada — erros de digitação e protocolos não encontrados acontecem todos os dias em um call center.

1. O que observei/coletei. Os três cenários acima, descritos a partir da rotina real de um painel de atendimento (busca linear sobre a lista de pedidos do dia).

2. O que a ferramenta sugeriu. Pedidos os três cenários a uma IAG para a mesma situação, ela descreveu o melhor caso como "o elemento buscado está na primeira posição" (correto) e o pior caso como "o elemento não existe na lista" (correto), mas descreveu o caso médio de forma genérica como "o elemento está em alguma posição aleatória", sem reconhecer que, na prática operacional descrita, a maioria das buscas de um atendente é por pedidos **abertos recentemente**, que tendem a estar mais próximos do início ou do fim da lista dependendo de como ela é ordenada — não uniformemente distribuídas.

3. O que eu decidi. Mantive a classificação teórica (A/B/C) da IAG, mas registrei o caso médio da IAG como uma simplificação que não reflete o padrão real de uso: a distribuição das buscas não é uniforme, é enviesada para os pedidos mais recentes.

4. Como validei minha decisão. Confrontei com a prática relatada por atendentes reais desse tipo de painel: a maior parte das consultas é sobre pedidos do próprio turno, o que enviesa a distribuição — validação qualitativa, por conhecimento do domínio, não por medição estatística formal (que exigiria logs reais de uso, fora do escopo deste exercício).

Você confiaria no resultado se o gestor tivesse testado só o melhor caso?

Não. Testar apenas o cenário A dá uma amostra de tamanho um, do cenário estatisticamente menos provável, e sistematicamente subestima o tempo de resposta que o usuário final vai perceber na maioria das interações — é o análogo, em desempenho, de aceitar corretude de um algoritmo com um único teste favorável (Exercício 3 da Aula 02). Comprar ou desenvolver um sistema com base só no melhor caso é decidir com os olhos fechados para o cenário que efetivamente vai se repetir todos os dias (o caso médio) e para o que vai gerar reclamação (o pior caso).

Qual cenário da IAG não representa bem a situação real?

O **caso médio**, pelo motivo já registrado: a IAG assume uma distribuição uniforme de onde o item buscado está na lista, mas o padrão real de uso de um painel de suporte é enviesado para itens recentes. Isso importa porque o caso médio é justamente o cenário que a aula recomenda validar por

análise empírica quando a matemática fica complexa — e aqui a complexidade não é matemática, é de conhecimento de domínio que só quem opera o sistema tem.

Exercício 4 — Duas soluções funcionam. Qual você escolheria?

Situação profissional escolhida

Detecção de duplicidade de clientes cadastrados (ex.: mesmo cliente cadastrado duas vezes com pequenas variações). **Algoritmo A**: comparação usando um índice/hash (O(n)). **Algoritmo B**: comparação de todo par de registros — o mesmo padrão do Selection Sort, $O(n^2)$.

n (registros)	Algoritmo A — O(n) (ms, escala 0,01 ms/registro)	Algoritmo B — $O(n^2)$ (ms, escala 0,01 ms/comparação)
100	1	100
1.000	10	10.000 (10 s)
10.000	100 (0,1 s)	1.000.000 (~16,7 min)
100.000	1.000 (1 s)	100.000.000 (~27,8 h)

Escala ilustrativa calibrada para deixar a comparação visível — a proporção entre as colunas é o que importa (n vs. n^2), não os valores absolutos, que dependem da implementação real.

1. O que observei/coletei. A tabela acima e o histórico de crescimento da base de clientes da situação hipotética escolhida (crescimento de 100 para 100.000 registros ao longo de alguns anos de operação).

2. O que a ferramenta sugeriu. Perguntado sobre a partir de que tamanho a diferença entre O(n) e $O(n^2)$ se torna crítica, uma IAG respondeu de forma genérica: "a diferença começa a importar a partir de milhares de registros", sem dar um número concreto atrelado ao contexto operacional (frequência de execução, janela de tempo disponível).

3. O que eu decidi. Refinei a resposta genérica da IAG com a resposta às cinco perguntas do enunciado:

- **Em qual tamanho a diferença começa a ser relevante?** Em torno de 10.000 registros, quando o Algoritmo B já ultrapassa minutos — inviável para rodar dentro de uma janela de batch noturna compartilhada com outras rotinas.
- **Em qual cenário eu escolheria A?** Sempre que a rotina precisa rodar com frequência (diária ou mais) e a base tende a crescer — que é exatamente o caso de detecção de duplicidade em produção.
- **Existe algum cenário em que B ainda seria aceitável?** Sim: uma limpeza pontual, única, em uma base pequena (poucas centenas de registros) e que roda uma única vez, sem repetição — o custo de desenvolver A não se paga para um uso descartável.
- **O custo de desenvolver A pode ser maior?** Sim — um algoritmo O(n) para detecção de duplicidade normalmente exige uma chave de comparação bem definida (normalização de dados, função de hash ou de blocking), que é trabalho de modelagem adicional que o algoritmo $O(n^2)$ ingênuo dispensa (ele compara tudo com tudo, sem precisar de nenhuma chave).
- **O custo de manutenção pode mudar minha decisão?** Sim, na direção oposta: se a lógica de normalização do Algoritmo A for frágil (muitas exceções, regras que mudam com frequência), o custo de manutenção pode superar o ganho de desempenho para bases que nunca crescem muito — mas essa não é a situação aqui, onde a base cresce.

Recomendação ao gerente (até 8 linhas)

Recomendo a solução A porque a base de clientes tende a crescer continuamente, e o Algoritmo B ($O(n^2)$) que hoje processa 100 registros em cerca de 100 ms passaria a levar quase 28 horas com 100.000 registros — um patamar que a empresa deve atingir em poucos anos de operação. O Algoritmo A (O(n)) mantém a mesma rotina em cerca de 1 segundo nesse mesmo volume. O custo adicional de

desenvolver A (definir uma chave de comparação normalizada) é pago uma única vez; o custo de manter B em produção cresce todo mês junto com a base de clientes, até se tornar inviável de rodar dentro da janela operacional disponível.

Exercício 5 — O código que parece simples pode esconder um problema

Código real escolhido

Escolhi o Selection Sort que implementei e medi para o Exercício 6 desta mesma lista (código real, não gerado pela IAG para este exercício).

```
static void selectionSort(double[] a) {
    int n = a.length;
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[min]) min = j;
        }
        double aux = a[i]; a[i] = a[min]; a[min] = aux;
    }
}
```

1. O que observei/coletei. Quantos laços existem: dois. **Quais estão aninhados:** o laço em `j` está aninhado dentro do laço em `i` — para cada valor de `i`, o laço em `j` executa por completo. **Como cada variável de controle evolui:** ambos incrementam de 1 em 1 (`i++`, `j++`), sem multiplicação nem divisão. **Qual é o tamanho da entrada n:** o comprimento do array `a` (número de preços/registros a ordenar).

2. Previsão (antes de consultar a IAG). Como os dois laços são de incremento simples e o interno depende do externo (de forma similar ao padrão "dois laços aninhados" do material, que corresponde a $O(n^2)$), esperei que multiplicar n por 10 multiplicasse o tempo de execução por aproximadamente 100 (10^2).

3. O que a ferramenta sugeriu (depois da minha previsão). Ao pedir à IAG para analisar o mesmo código, ela identificou corretamente $O(n^2)$ e, adicionalmente, apontou algo que eu não tinha registrado explicitamente: o laço externo vai até `n-2` (`i < n-1`) e não até `n-1`, porque o último elemento já está necessariamente na posição correta depois que todos os outros foram fixados — uma observação correta sobre a constante do laço, que não muda a classe assintótica, mas mostra atenção a um detalhe que eu tinha lido, mas não registrado como relevante.

4. Comparação: minha análise × análise da IAG. Concordamos na classificação ($O(n^2)$) e na estrutura dos dois laços aninhados. A IAG acrescentou a observação sobre o limite do laço externo, que eu verifiquei relendo o código e confirmando que é verdadeira e não afeta a complexidade — apenas evita uma iteração final desnecessária (uma otimização de constante, não de classe).

Confronto com o experimento real (Exercício 6)

Os tempos medidos no Exercício 6 (10.000 → 20.000 registros: 27,99 ms → 113,38 ms, um fator de **4,05x** para 2x o tamanho da entrada) confirmam a previsão de $O(n^2)$ feita antes de consultar a IAG: dobrar n multiplicou o tempo por aproximadamente 4 ($2^2 = 4$), como esperado para uma complexidade quadrática — a previsão "n vezes 10 → tempo vezes 100" da análise teórica se traduziu, na escala menor testada (2x), em "n vezes 2 → tempo vezes 4", exatamente a mesma relação (k^2 para um fator k de aumento de n).

Exercício 6 — Selection Sort: funciona, mas até quando?

Situação escolhida

Ordenar uma lista de preços de produtos de um catálogo, do menor para o maior.

1. O que observei/coletei. Implementei o Selection Sort em Java e medi o tempo de execução real (System.nanoTime()) para arrays de double gerados aleatoriamente, com tamanhos crescentes.

n	Tempo medido (ms)	Razão em relação ao tamanho anterior	Razão de tempo observada
100	0,1199	—	—
500	1,8774	5x	15,66x
1.000	2,4582	2x	1,31x
5.000	12,4710	5x	5,07x
10.000	27,9891	2x	2,24x
20.000	113,3797	2x	4,05x

Para os saltos maiores e mais estáveis (5.000→10.000 e 10.000→20.000, ambos com razão de tamanho 2x), a razão de tempo observada (2,24x e 4,05x) converge para o valor teórico esperado de $O(n^2)$ — $2^2 = 4$ —, com o salto 5.000→10.000 ainda mostrando influência de warm-up da JVM. Todos os arrays foram verificados como corretamente ordenados ao final (`ordenado=true` em todas as execuções).

2. O que a ferramenta sugeriu. Pedida uma previsão teórica, a IAG confirmou $O(n^2)$ e sugeriu que, para validar experimentalmente, os saltos de tamanho fossem sempre por um fator fixo (ex.: sempre dobrando), para que a razão esperada (o quadrado do fator) fosse mais fácil de comparar diretamente com a razão medida — sugestão que incorporei nos dois últimos pontos da tabela.

3. O que eu decidi. Considerei o crescimento observado **compatível com $O(n^2)$**, com a ressalva de que as primeiras medições (100 a 1.000) são pouco confiáveis por conta do tempo de warm-up da JVM (compilação just-in-time), sendo os pontos de 5.000 em diante os mais representativos do comportamento assintótico real.

4. Como validei minha decisão. Repeti a checagem de ordenação programaticamente (não visualmente) para cada tamanho testado, e usei dois saltos consecutivos com o mesmo fator de crescimento (2x) para comparar diretamente com a previsão teórica (4x) — a convergência da razão observada (2,24x, depois 4,05x) para o valor teórico à medida que n cresce é a evidência de que o ruído de medição perde peso relativo e o comportamento assintótico domina, exatamente como esperado.

Respostas às perguntas do enunciado

1. O crescimento observado parece compatível com $O(n^2)$? Sim, especialmente nas faixas maiores, como detalhado acima.

2. Em que momento percebi uma diferença significativa? Entre 10.000 e 20.000 registros: o tempo mais que quadruplicou (27,99 ms → 113,38 ms), e essa é a faixa em que uma rotina de ordenação síncrona já deixa de ser imperceptível para o usuário.

3. O computador apresentou exatamente o comportamento esperado? Não exatamente — os saltos pequenos (100→500→1.000) mostraram razões de tempo bem diferentes da teoria pura, por efeito de warm-up e de outras cargas do sistema operacional durante a medição.

4. Por que o resultado experimental não precisa ser uma curva perfeita? Porque a notação O é uma cota assintótica, válida "para todo n ≥ n0" — ela não faz nenhuma promessa sobre o comportamento em

valores pequenos de n, nem sobre fatores externos à contagem de operações (JIT, GC, agendamento do sistema operacional, cache de CPU). A teoria prevê a tendência de fundo, não cada ponto individual.

5. Você usaria esse algoritmo com 1 milhão de registros? Não, e a justificativa não é "O(n^2) é ruim" em abstrato: extrapolando a curva medida (113,38 ms para 20.000), o tempo estimado para 1.000.000 de registros (50x maior) seria da ordem de 50^2 = 2.500 vezes maior, ou seja, aproximadamente 283 segundos (quase 5 minutos) só para ordenar — inviável para uma operação que qualquer usuário espera ver responder em menos de um segundo, e desnecessário, já que `Collections.sort`/`Arrays.sort` (O(n log n)) resolveria o mesmo problema em uma fração desse tempo.

Exercício 7 — A IAG recomendou o algoritmo errado?

Situação realista escolhida

Endpoint de autocomplete de busca de produtos em um e-commerce, chamado a cada tecla digitada pelo usuário na barra de pesquisa.

- **O que representa n:** o número de produtos no catálogo elegíveis para sugestão.
- **Tamanho atual:** cerca de 50.000 produtos ativos.
- **Quanto pode crescer:** a empresa projeta 500.000 produtos em 3 anos (expansão de marketplace).
- **Frequência de execução:** muito alta — a cada tecla digitada, por milhares de usuários simultâneos (potencialmente dezenas de chamadas por segundo).
- **Restrições de memória:** o serviço roda em um contêiner com limite de memória definido (não é elástico à vontade).
- **Necessidade de resposta imediata:** sim — o usuário percebe qualquer atraso acima de ~100 ms como "travando" enquanto digita.

1. O que observei/coletei. As seis características acima do cenário de autocomplete, e as três complexidades candidatas: A O(1), B O(n), C O(n^2).

2. O que a ferramenta sugeriu. Confrontada com a afirmação "a solução A é obviamente a melhor", uma IAG (corretamente) não aceita a conclusão sem contexto, e pergunta de volta o que a solução A representa na prática — porque O(1) não é gratuito: geralmente significa uma estrutura pré-computada (ex.: um índice invertido ou um cache de sugestões populares), que tem custo de memória e de manutenção que não aparece na notação de tempo.

3. O que eu decidi. Não aceitei a conclusão "A é obviamente melhor" pelos motivos que o próprio enunciado antecipa: dado que a frequência de execução é altíssima e a necessidade de resposta é imediata, o tempo de resposta É o requisito dominante — então A (O(1)) de fato tende a vencer aqui, mas **não "obviamente"**: a decisão depende de A caber dentro da restrição de memória do contêiner (um índice O(1) para 500.000 produtos pode não caber se for ingênuo, ex.: uma tabela de hash para cada combinação de prefixo de busca) e do custo de manter esse índice atualizado a cada novo produto cadastrado (uma escrita cara pode não valer a pena se o catálogo mudar com muita frequência).

4. Como validei minha decisão. Apliquei o mesmo raciocínio do Exercício 4 da Aula 02 (custo de manter uma estrutura vs. ganho na leitura): para autocomplete, o padrão de mercado real (Elasticsearch, trie de prefixos, OpenSearch "completion suggester") de fato usa estruturas próximas de O(1)/O(log n) por consulta, confirmando que a escolha por A é acertada neste cenário específico — mas a validação foi checar as restrições (memória, custo de atualização), não aceitar "O(1) é sempre melhor" como regra universal.

Exercício 8 — Duas soluções prontas, uma matriz de decisão

Reaproveitando o cenário do autocomplete do Exercício 7, com a complicação adicional do enunciado: A (O(1), o índice pré-computado) tem maior custo de implementação; B (O(n), varrer o catálogo filtrando por prefixo a cada busca) é simples e barata; C (O(n^2), comparar a busca contra todo par produto×sugestão já sugerida para deduplicar) já está pronta e funcionando (um protótipo herdado).

Critério	A — O(1)	B — O(n)	C — O(n^2)
Desempenho	5	3	1
Custo	2	4	5
Escalabilidade	5	2	1
Complexidade de implementação	2	4	5 (já pronta)
Manutenção	3	4	2
Adequação ao problema	5	3	1

Justificativa de três notas

Escalabilidade de A = 5: um índice pré-computado consultado em O(1) não se degrada quando o catálogo cresce de 50.000 para 500.000 produtos — é exatamente o requisito de crescimento levantado no Exercício 7.

Complexidade de implementação de C = 5 ("já pronta"): a nota alta aqui não mede qualidade técnica, mede o critério específico pedido (esforço de implementação); é o único critério em que o algoritmo pior assintoticamente vence, o que é exatamente o ponto do exercício.

Desempenho de C = 1: O(n^2) sobre um catálogo de 50.000 produtos, executado a cada tecla digitada, é comparar até 2,5 bilhões de pares por requisição — tecnicamente inviável na prática, mesmo que "funcione" em um teste com poucos itens.

Peça à IAG: qual das três você escolheria? A IAG recomendou a solução A, com a justificativa de que o requisito de resposta imediata sob alta frequência domina os demais critérios.

Tentativa de convencer a IAG de que a escolha está errada. Argumentei que, como C "já está pronta e funcionando", o custo de trocar por A é desnecessário no curto prazo, e que o time deveria priorizar entregar valor mais rápido usando C. A IAG manteve a recomendação por A, mas qualificou a resposta: se o volume de tráfego atual for baixo o suficiente para C não causar timeout perceptível, adotar C **temporariamente** e migrar para A antes do crescimento projetado é uma sequência aceitável — mas manter C permanentemente, sabendo que o tráfego vai crescer, é a decisão que ela considerou errada, não a ideia de usar C como ponte de curto prazo.

Recomendação final

Eu escolheria a **solução A** porque o autocomplete é executado com frequência altíssima e o requisito de latência é rígido (abaixo de ~100 ms), e a solução C (O(n^2)) coloca o serviço em risco concreto de timeout assim que o catálogo ultrapassar a faixa de dezenas de milhares de itens que já é a realidade atual.

Em que circunstância eu mudaria a decisão?

Mudaria para C temporariamente se houvesse uma urgência de negócio inadiável (ex.: lançamento em poucos dias) e o catálogo estivesse momentaneamente pequeno o bastante (poucos milhares de itens) para C não travar visivelmente — mas com um plano de migração para A já aprovado e prazo definido, não como solução definitiva.

Exercício 8 (Fibonacci) — Três algoritmos, três decisões

Etapa 1 — Previsão

Antes de executar, minha expectativa era: **fibonacciConstante (O(1), fórmula de Binet)** seria o mais rápido, seguido por **fibonacciLinear (O(n))**, com **fibonacciQuadratico (O(n^2))** muito atrás para n grande — e que, para n muito grande, a diferença entre o O(1) e os demais deixaria de ser "um pouco mais rápido" e passaria a ser "o único que continua respondendo em tempo constante enquanto os outros demoram cada vez mais".

Etapa 2 — Experimento (execução real, Java 21, System.nanoTime())

```
static long fibonacciLinear(int n) {
    if (n == 0 || n == 1) return n;
    long a = 0, b = 1, c = 0;
    for (int i = 2; i <= n; i++) { c = a + b; a = b; b = c; }
    return c;
}
static long fibonacciQuadratico(int n) {
    if (n == 0 || n == 1) return n;
    long f = 0;
    for (int i = 2; i <= n; i++) {
        f = 0;
        for (int j = 1; j <= 2; j++) f = f + fibonacciLinear(i - j);
    }
    return f;
}
static long fibonacciConstante(int n) {
    double phi = (1 + Math.sqrt(5)) / 2, psi = (1 - Math.sqrt(5)) / 2;
    return Math.round((Math.pow(phi, n) - Math.pow(psi, n)) / Math.sqrt(5));
}
```

n	Algoritmo 1 — linear (ms)	Algoritmo 3 — constante/Binet (ms)
1.000	0,01432	0,00083
5.000	0,01961	0,00029
10.000	0,04110	0,00037
20.000	0,00670	0,00027
40.000	0,01233	0,00029
80.000	0,02481	0,00072

n	Algoritmo 2 — quadrático (ms)
200	0,1253
400	0,0826
800	0,2469
1.600	0,8436
3.200	3,1633

Note a diferença de escala de n necessária para observar cada algoritmo: o quadrático precisou de valores de n **25 vezes menores** (200 a 3.200, contra 1.000 a 80.000) para produzir tempos na mesma ordem de grandeza — exatamente o retrato prático de O(n^2) crescer muito mais rápido que O(n).

Etapa 3 — Confronto: previsão × observado

O Algoritmo 3 (constante) manteve-se sistematicamente abaixo de 0,001 ms em toda a faixa testada, **sem crescer perceptivelmente** de n=1.000 até n=80.000 — confirmando O(1). O Algoritmo 1 (linear) mostrou tempos maiores, mas com bastante ruído entre medições próximas (n=10.000 mediu mais que

n=20.000!) — nesta faixa de n, o custo de cada iteração do laço é tão pequeno que efeitos de warm-up da JVM dominam a medição; a tendência linear só fica visível de forma confiável olhando a ordem de grandeza ao longo de faixas bem mais amplas, ou usando ferramentas de microbenchmark dedicadas (JMH), que eliminam esse ruído. O Algoritmo 2 (quadrático) foi o único cuja curva de crescimento é visível a olho nu mesmo em n pequeno (200 a 3.200): o tempo cresce de forma muito mais acentuada que os outros dois, como esperado.

Etapa 4 — Investigação: por que o teoricamente mais eficiente pode não ser sempre o mais rápido na prática?

Pergunta feita à IAG e resposta obtida: "o comportamento assintótico só se manifesta claramente para n suficientemente grande; para n pequeno, custos constantes (overhead de chamada de função, cálculos de ponto flutuante mais caros que operações inteiras, efeitos de cache) podem fazer um algoritmo assintoticamente pior parecer mais rápido na prática."

Afirmação que precisei verificar (e que encontrei sozinho, executando o experimento, não apenas aceitando a resposta da IAG): o Algoritmo 3 usa `Math.pow` e aritmética de ponto flutuante com `√5`, que é mais cara **por operação** do que uma soma de inteiros do Algoritmo 1 — mas, mesmo assim, o Algoritmo 3 venceu em todos os tamanhos testados, porque "mais caro por operação" não superou "apenas 3 operações" (O(1) de verdade) contra "n operações" (O(n)) mesmo para n=1.000. **A afirmação da IAG está correta em geral, mas o experimento mostrou um limite dela**: eu verifiquei matematicamente que, para o Algoritmo 3 realmente perder do Algoritmo 1, o custo fixo de `Math.pow` precisaria ser maior que n operações de soma — algo que só aconteceria para valores de n extremamente pequenos (n < 10, aproximadamente), fora da faixa que testei.

Uma segunda limitação, encontrada por mim ao checar a corretude dos valores, não apenas o tempo: a fórmula de Binet perde precisão numérica bem antes de qualquer overflow de `long` acontecer. Medido diretamente:

n	fibonacciLinear(n) (exato, long)	fibonacciConstante(n) (Binet, double)
70	190392490709135	190392490709135 (correto)
90	2880067194370816120	2880067194370824704 (diverge — erro de arredondamento)
93 (overflow do long)	-6246583658587674878 (overflow, incorreto)	9223372036854775807 (saturado no limite do long, também incorreto)

Ou seja: **o Algoritmo 3 (O(1)) já retorna um resultado numericamente incorreto a partir de n=90, antes mesmo do Algoritmo 1 estourar o tipo `long` em n=93.** É exatamente o tipo de afirmação ("O(1) é sempre a melhor escolha") que precisa ser verificada e que, verificada, se revela falsa para este problema específico se a aplicação exigir valores exatos de Fibonacci além de n≈90.

Etapa 5 — Decisão

Não escolheria automaticamente o algoritmo O(1). A decisão depende de pelo menos dois fatores além da velocidade: (1) se a aplicação precisa de valores exatos de Fibonacci para n grande, o Algoritmo 3 está desqualificado por precisão numérica, restando escolher entre o Algoritmo 1 (correto até o limite do tipo usado) ou uma representação de precisão arbitrária (ex.: `BigInteger` em Java, trocando velocidade por exatidão); (2) se a aplicação só precisa de uma aproximação rápida (ex.: estimar a ordem de grandeza), o Algoritmo 3 continua sendo a melhor escolha.

Parecer técnico para um gestor não técnico (até 15 linhas)

Testamos três formas de calcular o mesmo resultado (a sequência de Fibonacci) e medimos o tempo real de cada uma à medida que o tamanho do problema cresce. Olhar apenas o tempo de hoje é

arriscado porque um processo que parece rápido com poucos dados pode se tornar impraticável quando o volume cresce — e o quanto ele piora depende da forma como o algoritmo foi construído, não apenas da velocidade do computador. Chamamos isso de "crescimento assintótico": a tendência do tempo de execução conforme os dados aumentam, não o valor absoluto medido hoje. Na nossa comparação, a solução mais sofisticada (uma fórmula matemática direta) manteve-se instantânea mesmo com dados 80 vezes maiores, enquanto a solução mais ingênua (comparar tudo com tudo) ficou visivelmente mais lenta com um aumento de apenas 16 vezes nos dados. Ainda assim, a solução mais rápida não é automaticamente a melhor: descobrimos que ela perde precisão para números muito grandes, um problema que a solução mais lenta não tem. A lição prática é que a escolha do algoritmo certo depende do contexto de uso (velocidade necessária, exatidão exigida, frequência de execução, volume esperado de dados) — diferentes algoritmos podem ser certos para diferentes situações, mesmo resolvendo, na aparência, o mesmo problema.